

Lab 2.1: C Libraries, Linking, and ELF Executable Structure

Lab Report and Knowledge Check

I. Introduction

When a C program that calls library functions such as `printf()`, `getpid()` and `time()` is compiled and run, several tools cooperate. The compiler turns the source into machine instructions, the linker resolves references to external libraries such as `libc`, and the operating system reads the resulting binary and maps it into virtual memory. This report documents the practical work completed for Lab 2 and answers three questions:

- How does a C program link to the C Standard Library?
- Where do the C library headers and the compiled library code physically live on disk?
- What is the difference between static and dynamic linking, and how can the resulting executables be inspected?

A small program, `procinfo.c`, was written to use several library functions. It was built twice (statically and dynamically), both builds were run, and the dynamic build was inspected with `readelf` and `ldd`. All work was done in a Linux terminal on the host `abhiyan@nepal`, which is a 64-bit ARM (AArch64) machine.

II. A Practical C Program: Process Information

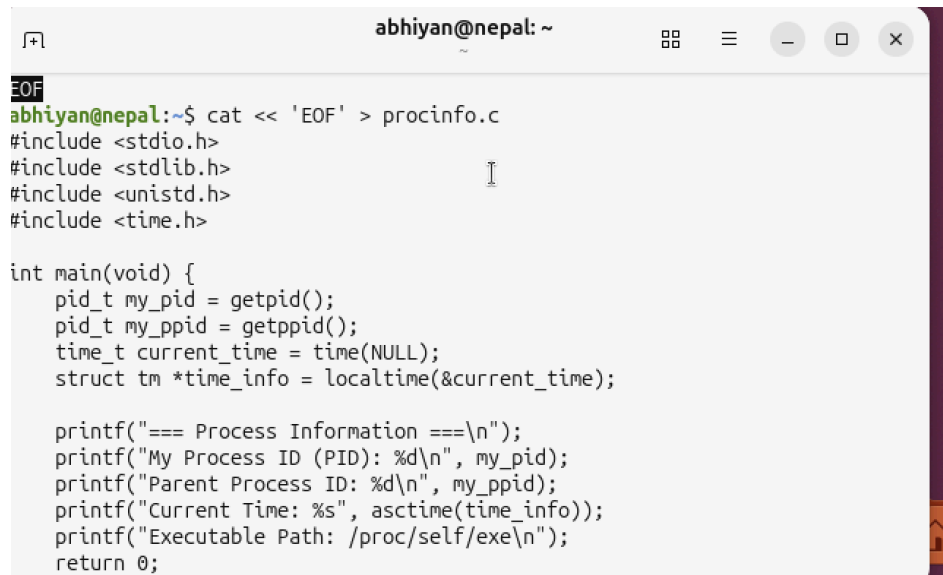
A. The Program

The program `procinfo.c` reports its own process ID, its parent process ID and the current local time. It uses `getpid()` and `getppid()` from `<unistd.h>`, `time()` and `localtime()` from `<time.h>`, and `printf()` and `asctime()` from the C library. All of these are implemented in `libc`, which is why linking matters. The source file was created from the terminal with a here-document (`cat << 'EOF' > procinfo.c`).

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");
    return 0;
}
```

A terminal window titled 'abhiyan@nepal: ~' showing the creation of a C file named 'procinfo.c' using a here-document. The user enters 'cat << 'EOF' > procinfo.c' followed by several include statements and a main function that prints process information.

```
abhiyan@nepal:~$ cat << 'EOF' > procinfo.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

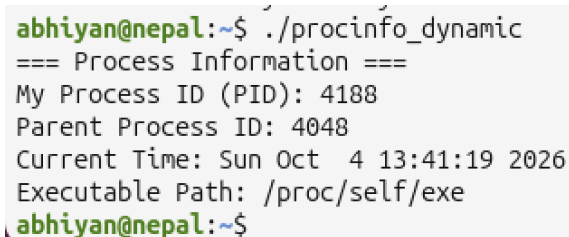
    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");
    return 0;
}
EOF
```

Figure 1. `procinfo.c` created in the terminal with a here-document.

B. Compiling and Running

The program was compiled normally (dynamic linking is the default) and then run. The output shows the process ID (4188), its parent (4048, the shell that launched it) and the local time.

```
gcc procinfo.c -o procinfo_dynamic
./procinfo_dynamic
```

A terminal window showing the execution of the compiled program 'procinfo_dynamic'. The output displays process information including PID, PPID, current time, and executable path.

```
abhiyan@nepal:~$ ./procinfo_dynamic
=== Process Information ===
My Process ID (PID): 4188
Parent Process ID: 4048
Current Time: Sun Oct  4 13:41:19 2026
Executable Path: /proc/self/exe
abhiyan@nepal:~$
```

Figure 2. Running `procinfo_dynamic`.

III. The C Standard Library: Two Halves

The C Standard Library (libc, implemented on Linux by glibc) is split into two physical components that serve different purposes.

A. Part 1: Header Files — The Construction Manual

Header files (.h) hold declarations only: function prototypes, macros, typedefs and structure definitions. They tell the compiler what functions exist and how to call them, but contain no machine code. They live in `/usr/include/`. When a program uses `#include <stdio.h>`, the preprocessor copies the header into the source before compilation, so the compiler knows `printf()` exists and what arguments it takes, but not where its code is.

B. Part 2: Precompiled Machine Code — The Actual Libraries

The real machine instructions for `printf()`, `getpid()`, `time()` and `malloc()` are stored in library files, which come in two forms:

- **Static libraries (.a):** archives of object files; the linker copies the needed code into the executable.
- **Shared libraries (.so):** standalone binaries loaded at run time by the dynamic linker, so many programs share one copy in RAM.

C. Verifying the Locations on This System

```
ls /usr/include/stdio.h /usr/include/unistd.h
ls /usr/lib/aarch64-linux-gnu/libc.so* /usr/lib/aarch64-linux-gnu/libc.a
```

```
-----
Executable Path: /proc/self/exe
abhiyan@nepal:~$ ls /usr/include/stdio.h /usr/include/unistd.h
ls /usr/lib/aarch64-linux-gnu/libc.so* /usr/lib/aarch64-linux-gnu/libc.a
/usr/include/stdio.h /usr/include/unistd.h
/usr/lib/aarch64-linux-gnu/libc.a /usr/lib/aarch64-linux-gnu/libc.so.6
/usr/lib/aarch64-linux-gnu/libc.so
abhiyan@nepal:~$
```

Figure 3. Locations of the C headers, the static library and the shared library.

The output confirms both halves. The headers `stdio.h` and `unistd.h` are in `/usr/include/`. The static library `libc.a` and the shared library `libc.so.6` (plus the `libc.so` development name) are in `/usr/lib/aarch64-linux-gnu/`. The handout shows `/lib/x86_64-linux-gnu/` because it was written for an x86-64 machine; the directory name contains the CPU architecture, and this system is 64-bit ARM (aarch64).

Table 1. Two parts of the C Standard Library (as found on this system)

Component	Contains	Location
Headers (.h)	Declarations, prototypes, macros	<code>/usr/include/</code>
Static library (.a)	Precompiled object code (archive)	<code>/usr/lib/aarch64-linux-gnu/libc.a</code>
Shared library (.so)	Precompiled position-independent code	<code>/usr/lib/aarch64-linux-gnu/libc.so.6</code>

IV. Static vs. Dynamic Linking

Linking resolves symbolic references (such as the name “printf”) to real addresses. The time and method of that resolution separate the two approaches.

A. Building Both Versions

```
gcc procinfo.c -o procinfo_dynamic
gcc -static procinfo.c -o procinfo_static
ls -lh procinfo_dynamic procinfo_static
```

```

COR
abhiyan@nepal:~$ gcc procinfo.c -o procinfo_dynamic
gcc -static procinfo.c -o procinfo_static
ls -lh procinfo_dynamic procinfo_static
-rwxrwxr-x 1 abhiyan abhiyan 70K Oct  4 13:40 procinfo_dynamic
-rwxrwxr-x 1 abhiyan abhiyan 769K Oct  4 13:40 procinfo_static
abhiyan@nepal:~$

```

Figure 4. Compiling the dynamic and static versions and comparing file sizes.

The static executable (769 KB) is roughly eleven times larger than the dynamic one (70 KB), because the linker copied the required libc code from the .a archive into the file. The exact sizes differ from the handout's example (850 KB vs 16 KB) because this is an ARM64 system with a different toolchain, but the pattern is identical.

B. Running the Static Version

```
./procinfo_static
```

```

section header string table index: 28
abhiyan@nepal:~$ ./procinfo_static
=== Process Information ===
My Process ID (PID): 8523
Parent Process ID: 3555
Current Time: Sun Oct  4 16:56:13 2026
Executable Path: /proc/self/exe
abhiyan@nepal:~$

```

Figure 5. Running procinfo_static.

The static binary prints exactly the same kind of output as the dynamic one (PID 8523, parent 3555), so the way the program is linked changes how the code is packaged but not what it does. The difference is that this file needs no shared library to run.

Table 2. Static vs. dynamic linking

Aspect	Static linking	Dynamic linking
When code is added	At link time, copied from .a files	At run time, loaded from .so files
Measured size	769 KB (procinfo_static)	70 KB (procinfo_dynamic)
Needs .so files to run	No, self-contained	Yes, fails if one is missing
ldd result	"not a dynamic executable"	Lists libc.so.6 and the dynamic linker
Library updates	Program must be recompiled	Automatic for all programs
Memory use	Each program carries its own copy	One copy shared in RAM
Typical use	Embedded systems, containers, portability	Default on modern Linux

V. Inspecting Executables with readelf

Linux executables use the ELF (Executable and Linkable Format) binary format. readelf reads the metadata of an ELF file without running it. Three views were used: the ELF header (-h), the program headers (-l) and the dynamic section (-d).

A. View 1: The ELF Header (readelf -h)

```
readelf -h procinfo_dynamic | head -20  
readelf -h procinfo_dynamic
```

```
abhiyan@nepal:~$ readelf -h procinfo_dynamic | head -20  
ELF Header:  
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00  
  Class:                               ELF64  
  Data:                               2's complement, little endian  
  Version:                             1 (current)  
  OS/ABI:                               UNIX - System V  
  ABI Version:                           0  
  Type:                                  DYN (Position-Independent Executable file  
)  
  Machine:                              AArch64  
  Version:                              0x1  
  Entry point address:                   0x900  
  Start of program headers:              64 (bytes into file)  
  Start of section headers:              68944 (bytes into file)  
  Flags:                                 0x0  
  Size of this header:                   64 (bytes)  
  Size of program headers:                56 (bytes)
```

Figure 6. ELF header of procinfo_dynamic (first part).

```
  Data:                               2's complement, little endian  
  Version:                             1 (current)  
  OS/ABI:                               UNIX - System V  
  ABI Version:                           0  
  Type:                                  DYN (Position-Independent Executable file  
)  
  Machine:                              AArch64  
  Version:                              0x1  
  Entry point address:                   0x900  
  Start of program headers:              64 (bytes into file)  
  Start of section headers:              68944 (bytes into file)  
  Flags:                                 0x0  
  Size of this header:                   64 (bytes)  
  Size of program headers:                56 (bytes)  
  Number of program headers:              10  
  Size of section headers:                64 (bytes)  
  Number of section headers:              29  
  Section header string table index: 28  
abhiyan@nepal:~$
```

Figure 7. ELF header of procinfo_dynamic (remaining fields).

Table 3. Key fields of the ELF header

Field	Value	Meaning
Magic	7f 45 4c 46 02 01 01 00 ...	".ELF" in ASCII: a valid ELF file
Class	ELF64	64-bit executable
Data	2's complement, little endian	Byte order of the CPU
OS/ABI	UNIX - System V	Standard Unix ABI
Type	DYN (Position-Independent Executable)	PIE; the base address can be randomised by the OS
Machine	AArch64	Built for 64-bit ARM; will not run on x86-64
Entry point address	0x900	Offset where execution starts (relative to the load base)

Field	Value	Meaning
Start of program headers	64 bytes into file	Directly after the 64-byte ELF header
Number of program headers	10	Segments described for the loader
Number of section headers	29	Sections (.text, .data, .dynamic, ...)

B. View 2: The Program Headers (readelf -l)

The program headers tell the OS loader which segments to map into memory and with what permissions.

```
readelf -l procinfo_dynamic
```

INTERP	0x0000000000000294	0x0000000000000294	0x0000000000000294	
	0x00000000000001b	0x00000000000001b	R	0x1
[Requesting program interpreter: /lib/ld-linux-aarch64.so.1]				
LOAD	0x0000000000000000	0x0000000000000000	0x0000000000000000	
	0x0000000000000cbc	0x0000000000000cbc	R E	0x10000
LOAD	0x000000000000fd40	0x000000000001fd40	0x000000000001fd40	
	0x0000000000002d0	0x0000000000002d8	RW	0x10000
DYNAMIC	0x000000000000fd50	0x000000000001fd50	0x000000000001fd50	
	0x000000000000200	0x000000000000200	RW	0x8
NOTE	0x000000000000270	0x000000000000270	0x000000000000270	
	0x000000000000024	0x000000000000024	R	0x4
NOTE	0x000000000000c9c	0x000000000000c9c	0x000000000000c9c	
	0x000000000000020	0x000000000000020	R	0x4
GNU_EH_FRAME	0x000000000000ba8	0x000000000000ba8	0x000000000000ba8	
	0x00000000000003c	0x00000000000003c	R	0x4
GNU_STACK	0x0000000000000000	0x0000000000000000	0x0000000000000000	
	0x0000000000000000	0x0000000000000000	RW	0x10
GNU_RELRO	0x000000000000fd40	0x000000000001fd40	0x000000000001fd40	
	0x0000000000002c0	0x0000000000002c0	R	0x1

Figure 8. Program headers of procinfo_dynamic (INTERP through GNU_RELRO).

Reading the output:

- **INTERP:** “Requesting program interpreter: /lib/ld-linux-aarch64.so.1”. The kernel loads this dynamic linker first, and it then loads libc and starts the program.
- **LOAD (R E):** the first LOAD segment starts at offset 0, is 0xcbc bytes long and is readable and executable. This is the machine code (.text) together with read-only data.
- **LOAD (RW):** the second LOAD segment holds the writable data. Its memory size (0x2d8) is 8 bytes larger than its file size (0x2d0); the extra bytes are the BSS, which exists only in memory and is zero-filled.
- **Alignment 0x10000:** both LOAD segments are aligned to 64 KiB. This padding is a likely reason why even this tiny program is about 70 KB on disk.
- **DYNAMIC:** at offset 0xfd50, the same address that readelf -d reports for the dynamic section (Figure 9), which cross-checks the two views.
- **GNU_STACK (RW):** the flags are RW and not RWE, so the stack is not executable. This is the secure default that blocks stack-based code injection.
- **GNU_RELRO:** a region that is made read-only after relocation, which hardens the global offset table against overwriting.

- **NOTE and GNU_EH_FRAME:** build metadata and exception-unwinding data; they do not affect how the program runs.

The header reported 10 program headers; the screenshot shows nine of them (INTERP to GNU_RELRO), and the tenth, PHDR, is the line above INTERP.

C. View 3: The Dynamic Section (readelf -d)

```
readelf -d procinfo_dynamic | head -12
readelf -d procinfo_dynamic
```

```
abhiyan@nepal:~$ readelf -d procinfo_dynamic | head -12

Dynamic section at offset 0xfd50 contains 28 entries:
  Tag                Type              Name/Value
  0x0000000000000001 (NEEDED)           Shared library: [libc.so.6]
  0x0000000000000001 (NEEDED)           Shared library: [ld-linux-aarch64.so.
1]
  0x000000000000000c (INIT)              0x7e0
  0x000000000000000d (FINI)              0xaf8
  0x0000000000000019 (INIT_ARRAY)          0x1fd40
  0x000000000000001b (INIT_ARRAYSZ)       8 (bytes)
  0x000000000000001a (FINI_ARRAY)         0x1fd48
  0x000000000000001c (FINI_ARRAYSZ)       8 (bytes)
  0x000000006ffffef5 (GNU_HASH)          0x2b0
abhiyan@nepal:~$
```

Figure 9. Top of the dynamic section, showing the NEEDED libraries.

The dynamic section is at offset 0xfd50 and has 28 entries. The most important lines are the two NEEDED entries: libc.so.6 (the C library) and ld-linux-aarch64.so.1 (the dynamic linker). The dynamic linker maps every NEEDED library into the process before main() runs, and the program will refuse to start if one is missing. The other entries in Figure 9 give the addresses of the initialisation function (INIT 0x7e0), the termination function (FINI 0xaf8) and their arrays.

```
0x0000000000000006 (SYMTAB)              0x2d0
0x000000000000000a (STRSZ)              243 (bytes)
0x000000000000000b (SYMENT)             24 (bytes)
0x0000000000000015 (DEBUG)              0x0
0x0000000000000003 (PLTGOT)            0x1ff50
0x0000000000000002 (PLTRELSZ)          288 (bytes)
0x0000000000000014 (PLTREL)            RELA
0x0000000000000017 (JMPREL)            0x6c0
0x0000000000000007 (RELA)              0x5e8
0x0000000000000008 (RELASZ)            216 (bytes)
0x0000000000000009 (RELAENT)           24 (bytes)
0x000000000000001e (FLAGS)             BIND_NOW
0x000000006fffffb (FLAGS_1)           Flags: NOW PIE
0x000000006fffffe (VERNEED)           0x598
0x000000006ffffff (VERNEEDNUM)         2
0x000000006fffff0 (VERSYM)            0x574
0x000000006fffff9 (RELACOUNT)          4
0x0000000000000000 (NULL)             0x0
abhiyan@nepal:~$
```

Figure 10. Lower part of the dynamic section of procinfo_dynamic.

The remaining entries describe how the dynamic linker must prepare the program:

- **SYMTAB / STRSZ / SYMENT:** location of the dynamic symbol table (0x2d0), the size of the string table (243 bytes) and the size of one symbol entry (24 bytes).

- **PLTGOT, PLTRELSZ, PLTREL, JMPREL:** the procedure linkage table and global offset table, used to call library functions such as printf() through relocation entries (288 bytes of RELA-type entries).
- **RELA, RELASZ, RELAENT, RELACOUNT:** the relocation table (216 bytes, 24 bytes per entry, 4 relative relocations) that the loader fixes up at start-up.
- **FLAGS = BIND_NOW and FLAGS_1 = NOW PIE:** all symbols are resolved at load time instead of lazily, and the executable is position-independent (PIE), matching Type DYN in the ELF header.
- **VERNEED / VERSYM:** symbol-version information required from libc.

VI. Tracing Dependencies with ldd

readelf -d shows only the names of the required libraries. ldd goes further: it invokes the dynamic linker to resolve every NEEDED entry to a full path on disk. It was run on both executables.

```
ldd procinfo_dynamic
ldd procinfo_static
```

```

XXXXXXXXXXXXXXXXXXXX (NULL) XXXX
abhiyan@nepal:~$ ldd procinfo_dynamic
ldd procinfo_static
linux-vdso.so.1 (0x0000e2778163f000)
libc.so.6 => /usr/lib/aarch64-linux-gnu/libc.so.6 (0x0000e277813f0000)
/lib/ld-linux-aarch64.so.1 (0x0000e277815f0000)
not a dynamic executable
abhiyan@nepal:~$
```

Figure 11. ldd on the dynamic executable, followed by ldd on the static executable.

What the output shows:

- **linux-vdso.so.1** is a virtual shared object injected by the kernel into every process; it provides fast system-call entry points and does not exist as a file on disk.
- **libc.so.6 => /usr/lib/aarch64-linux-gnu/libc.so.6** is the C library that supplies printf(), getpid(), time() and the other functions used by procinfo.c. It is the same file whose presence was verified in Figure 3.
- **/lib/ld-linux-aarch64.so.1** is the dynamic linker itself, the program that reads INTERP and NEEDED and loads everything before main() runs. (On x86-64 this is /lib64/ld-linux-x86-64.so.2.)
- **“not a dynamic executable”** is printed for procinfo_static. A static binary has no DYNAMIC section and no INTERP entry, so there are no run-time library dependencies.

VII. Command Reference Summary

Table 4. Inspecting ELF executables

Command	Look for	What it tells the OS
readelf -h <file>	Type, Machine	Is this a valid executable, and does it match the CPU architecture?
readelf -l <file>	INTERP	Which dynamic linker must run first to handle shared libraries?
readelf -l <file>	GNU_STACK	Is the stack secure (RW) or dangerously executable (RWE)?

Command	Look for	What it tells the OS
<code>readelf -l <file></code>	LOAD (flags)	Which regions are code (R E), read-only data (R) or read-write data (RW)?
<code>readelf -d <file></code>	NEEDED	Which shared libraries must be mapped before <code>main()</code> starts?
<code>ldd <file></code>	Full output	Where on the filesystem are the required <code>.so</code> files?
<code>ldd <file></code>	"not a dynamic executable"	The binary is statically linked with no run-time dependencies.

VIII. Knowledge Check Questions

Q: What is the fundamental difference between a C header file and a shared library?

A header file (`.h`) is source text containing only declarations: prototypes, macros, typedefs and structures. It is used by the preprocessor and compiler at compile time and holds no executable code; it lives in `/usr/include/`. A shared library (`.so`) is a compiled binary containing the actual machine code of the functions. It is used by the dynamic linker at run time and lives in `/lib/` or `/usr/lib/` (here `/usr/lib/aarch64-linux-gnu/libc.so.6`). The header tells the compiler how to call `printf()`; the shared library is where `printf()` actually runs.

Q: What is static and dynamic linking?

Linking is the step that resolves function names to real addresses. In static linking the linker copies the needed machine code from `.a` archives into the executable at compile time. The result is self-contained and portable, but large (769 KB for `procinfo_static`) and it must be rebuilt to pick up a library update; `ldd` reports it as "not a dynamic executable". In dynamic linking the executable stores only references to shared libraries and the dynamic linker loads the `.so` files when the program starts. The result is small (70 KB for `procinfo_dynamic`) and shares library code in RAM, but it needs the libraries to be present at run time (`readelf -d` lists `libc.so.6` as NEEDED). Dynamic linking is the default on Linux; static linking is chosen with `gcc -static`.

Q: What is the ELF header? Write down the commands to check headers.

The ELF header is the first structure in an ELF file. It carries global metadata: the magic number (7f 45 4c 46), the class (32- or 64-bit), byte order, ELF version, OS/ABI, file type (EXEC or DYN), target machine (CPU architecture), the entry-point address, and the locations and sizes of the program-header and section-header tables. The OS uses it to decide whether the file is a valid executable for this CPU. For `procinfo_dynamic` it shows ELF64, little endian, Type DYN (position-independent executable), Machine AArch64 and entry point 0x900 (Figures 6 and 7).

Commands:

```
readelf -h procinfo_dynamic    # ELF header
readelf -l procinfo_dynamic    # program headers (segments, INTERP, GNU_STACK)
readelf -S procinfo_dynamic    # section headers
readelf -d procinfo_dynamic    # dynamic section (NEEDED libraries)
readelf -a procinfo_dynamic    # everything at once
```

Q: What does bookmark mean? Provide an example.

The handout does not link “bookmark” to any topic in this lab, so the answer gives the general meaning. A bookmark is a saved, named pointer to a location that lets a user return to it quickly without searching again. Examples: a browser bookmark saved for the lab’s GitHub repository page; a bookmark in a Word document that jumps to a heading; or a shell shortcut such as saving a path in a variable (export LAB=~/.Batch38_LAB/Lab_2.1) and then using cd \$LAB.

Q: You are required to create proper documentation of your findings and push it to GitHub.

This report is the documentation. The work lives in the folder Lab_2.1 of the local repository /home/abhiyan/Batch38_LAB, whose remote is https://github.com/abhiyan0/batch38_ai_OS_abhiyansharma. The compiled binaries were excluded with a .gitignore file, and the work was committed and pushed to the main branch with these commands:

```
cd /home/abhiyan/Batch38_LAB/Lab_2.1
printf "procinfo_static\nprocinfo_dynamic\n" > .gitignore
git add .
git commit -m "Lab 2.1: C libraries, linking and ELF inspection"
git push -u origin main
```

```
branch 'main' set up to track 'origin/main'.
abhiyan@nepal:~/Batch38_LAB/Lab_2.1$ git config --unset-all branch.main.merge
git push -u origin main
Username for 'https://github.com': abhiyan0
Password for 'https://abhiyan0@github.com':
warning: branch.main.remote has multiple values
branch 'main' set up to track 'origin/main'.
Everything up-to-date
abhiyan@nepal:~/Batch38_LAB/Lab_2.1$ git pull --rebase origin main
git push -u origin main
From https://github.com/abhiyan0/batch38_ai_OS_abhiyansharma
 * branch          main          -> FETCH_HEAD
Already up to date.
Username for 'https://github.com': abhiyan0
Password for 'https://abhiyan0@github.com':
warning: branch.main.remote has multiple values
branch 'main' set up to track 'origin/main'.
Everything up-to-date
abhiyan@nepal:~/Batch38_LAB/Lab_2.1$
```

Figure 12. Pushing to GitHub: main now tracks origin/main and is up to date.

Git reported that branch main is set up to track origin/main and that everything is up to date, so the local main branch and the GitHub main branch are in sync. A later git pull --rebase origin main also reported “Already up to date”. To publish further files such as this report, copy them into Lab_2.1 and repeat git add ., git commit and git push.

IX. Lab Tasks Completed

1. Wrote procinfo.c using getpid(), getppid(), time(), localtime(), asctime() and printf() (Figure 1) and ran it (Figure 2).
2. Located the C headers and the static and shared libc on disk (Figure 3, Table 1).
3. Compiled the program dynamically and statically, compared the file sizes (70 KB against 769 KB) and ran the static binary (Figures 4 and 5, Table 2).

4. Inspected the ELF header with `readelf -h`: ELF64, little endian, DYN, AArch64 (Figures 6 and 7, Table 3).
5. Inspected the program headers with `readelf -l`: INTERP, LOAD segments, GNU_STACK and GNU_RELRO (Figure 8).
6. Inspected the dynamic section with `readelf -d` and found the NEEDED libraries (Figures 9 and 10).
7. Traced the shared-library dependencies with `ldd` and confirmed that the static binary is “not a dynamic executable” (Figure 11).
8. Answered all knowledge check questions and pushed the work to GitHub (Section VIII, Figure 12).

X. Conclusion

This lab showed how a C program that calls `printf()`, `getpid()` and `time()` depends on the C Standard Library, which has two parts: headers in `/usr/include/` that declare the functions, and precompiled code in `libc.a` and `libc.so.6` that implements them. Static linking produced a self-contained 769 KB binary, while dynamic linking produced a 70 KB binary that relies on `libc.so.6` and the dynamic linker at run time. `readelf` showed that the dynamic build is a 64-bit, little-endian, position-independent AArch64 executable that requests `/lib/ld-linux-aarch64.so.1`, has a non-executable stack and lists `libc.so.6` as a needed library. `ldd` resolved the dependencies to `/usr/lib/aarch64-linux-gnu/libc.so.6` and confirmed that the static build has none. The results and this report were pushed to GitHub.